

Volleyball Activity Analysis System

Object detection, trajectory prediction and natural-language scene understanding

Carlos Vicente · Javier Revuelta · Angel Moline · Jorge Molia

Neural Networks / Deep Learning — Universidad San Jorge

30 May 2026



Figure 1. Full pipeline output on the first frame: player detections with team and role labels, LSTM trajectory arrows, referee zones, and the 2D minimap in the bottom-right corner.

1. Project Overview

The goal of this project was to build a single application that analyses a volleyball match and combines, in one pipeline, the main neural network families covered during the course. We chose volleyball because the players are clearly separated into two teams by shirt colour, the ball is small and fast enough to make detection non-trivial, and it is one of the sports allowed by the assignment.

The system takes a video clip or a single image and returns an annotated frame with the players detected and tracked, their team and on-court role, a short-term trajectory prediction for each player, the ball position, referee detection, and a 2D top-view minimap. A vision-language model can answer questions about the scene in natural language. Everything runs from a single Gradio interface, so nothing needs to be called from a terminal.

Five trained components work together: a custom YOLOv8 model for the players, a second custom YOLOv8 for the ball, a convolutional network that classifies each player's role, a recurrent network that predicts movement, and a small fully-connected network that assigns players to a team from shirt colour. Gemma 4 via Unsloth handles the language layer without fine-tuning.

Model	Metric	Value	Params	Epochs
Player detector (YOLOv8s)	mAP@50	0.834	11.1 M	100
Ball detector (YOLOv8s)	mAP@50	0.942	11.1 M	80
Role classifier (CNN)	Val accuracy	93.6%	0.42 M	25
Trajectory model (LSTM)	Val MSE	0.00105	0.55 M	50
Team classifier (MLP)	Val accuracy	90.8%	1,123	40

Table 1. Summary of trained models and their validation metrics.

2. System Pipeline

For every frame the system runs the modules in a fixed order. First, the custom YOLO player detector finds all people on the court. Then CameraMotionEstimator (ORB features + RANSAC) estimates how much the broadcast camera has moved and adjusts the homography accordingly. Detections outside the court polygon are discarded, removing most bench, staff and spectator false positives. ByteTrack then links each detection to an existing track and assigns a persistent ID.

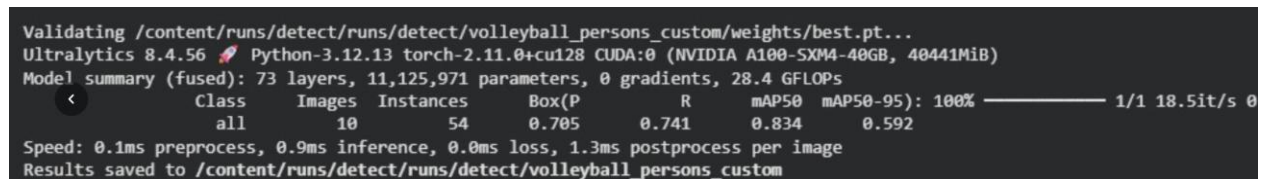
With the tracks in place, the pipeline classifies each player's team by projecting their foot position through the homography: players on the left of the net ($x < 225$ on the minimap canvas) are team 1, the rest are team 2. The colour MLP can override this when it is confident enough. The CNN then classifies the role from the player crop. The LSTM gets the last 20 frames of each track and predicts the next 10 positions, drawn as arrows on the output. In parallel, the ball detector finds the ball and updates a 40-frame trajectory buffer. Gemma 4 gets the annotated frame plus a structured summary of the detections every 60 frames.

3. Module Descriptions

3.1 Player Detector (YOLOv8)

We fine-tuned a YOLOv8s model (11.1 M parameters, 28.4 GFLOPs) on the public Roboflow dataset player.yolov8, which labels players by team. Training ran for 100 epochs at 640 px input with early stopping (patience 20). Because the dataset is not large, we applied offline augmentation with Albumentations (horizontal flips, brightness and hue jitter, small affine transforms) on top of YOLO's standard mosaic and mixup. The confidence threshold at inference is 0.35 and detections under 60 pixels tall are dropped.

On the validation set the model reached precision 0.705, recall 0.741, mAP@50 of 0.834 and mAP@50-95 of 0.592. The mAP@50 is solid for a dataset of this size. The drop at the stricter mAP@50-95 is expected, since tight box localisation suffers most when training data is limited.



```
Validating /content/runs/detect/runs/detect/volleyball_persons_custom/weights/best.pt...
Ultralytics 8.4.56 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (NVIDIA A100-SXM4-40GB, 40441MiB)
Model summary (fused): 73 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs


| Class | Images | Instances | Box(P) | R     | mAP50 | mAP50-95) |
|-------|--------|-----------|--------|-------|-------|-----------|
| all   | 10     | 54        | 0.705  | 0.741 | 0.834 | 0.592     |

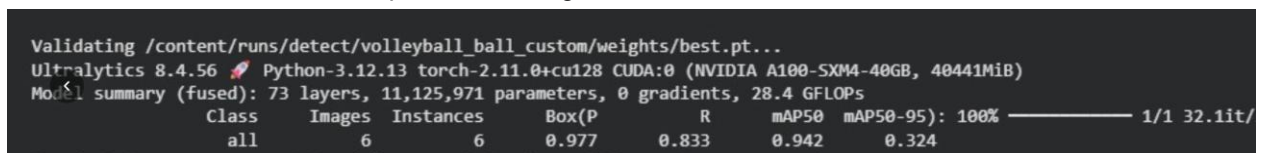

Speed: 0.1ms preprocess, 0.9ms inference, 0.0ms loss, 1.3ms postprocess per image
Results saved to /content/runs/detect/runs/detect/volleyball_persons_custom
```

Figure 2. Player detector validation results from the training notebook (mAP@50 = 0.834).

3.2 Ball Detector (YOLOv8)

The ball was the hardest part of the project. It is small, fast and often motion-blurred, and the standard COCO sports-ball class missed it almost completely on our footage. We built our own dataset by extracting frames from the match video and annotating the ball by hand. We then fine-tuned a separate YOLOv8s for 80 epochs at 640 px with patience 30, reducing rotation augmentation and disabling mixup since a ball has no orientation and mixing two frames hurts a single small target.

On the validation set the detector scored precision 0.977, recall 0.833, mAP@50 of 0.942 and mAP@50-95 of 0.324. The high mAP@50 confirms the detector finds the ball reliably. The low mAP@50-95 reflects loose box fit and a validation set that is small by design, since the footage we had was limited. On the actual match video the dedicated detector is far better than the COCO baseline, which was the whole point of building the dataset.



```
Validating /content/runs/detect/volleyball_ball_custom/weights/best.pt...
Ultralytics 8.4.56 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (NVIDIA A100-SXM4-40GB, 40441MiB)
Model summary (fused): 73 layers, 11,125,971 parameters, 0 gradients, 28.4 GFLOPs


| Class | Images | Instances | Box(P) | R     | mAP50 | mAP50-95) |
|-------|--------|-----------|--------|-------|-------|-----------|
| all   | 6      | 6         | 0.977  | 0.833 | 0.942 | 0.324     |


```

Figure 3. Ball detector validation results from the training notebook (mAP@50 = 0.942).

3.3 Role Classifier (CNN)

Each detected player crop goes through VolleyballCNN, a custom convolutional network with about 0.42 M parameters. It has four convolutional blocks (Conv-BatchNorm-ReLU-MaxPool, widening from 32 to 256 channels), followed by adaptive average pooling and a fully-connected head with dropout (p=0.5). It classifies the player into one of five roles: attacker, receiver, server, setter and referee.

We trained it on player crops cut from the volley_detection v5 dataset, resized to 224x224, with horizontal flips, colour jitter and small rotations as augmentation. The optimiser was AdamW

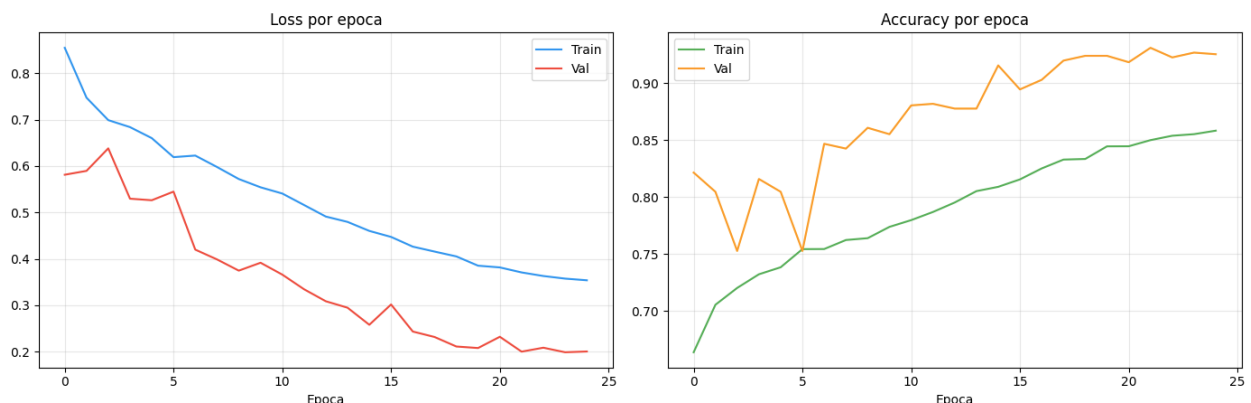
(lr=1e-3, weight decay 1e-4) with a cosine learning-rate schedule and cross-entropy loss over 25 epochs. Validation accuracy reached 93.1%. The same training loop is also available inside the Gradio GUI so the classifier can be retrained with different hyperparameters without leaving the application.

```

Epoca [ 1/25] Train Loss: 0.8551 Acc: 0.664 | Val Loss: 0.5814 Acc: 0.822
Epoca [ 5/25] Train Loss: 0.6603 Acc: 0.738 | Val Loss: 0.5264 Acc: 0.805
Epoca [10/25] Train Loss: 0.5543 Acc: 0.774 | Val Loss: 0.3916 Acc: 0.855
Epoca [15/25] Train Loss: 0.4603 Acc: 0.809 | Val Loss: 0.2579 Acc: 0.916
Epoca [20/25] Train Loss: 0.3853 Acc: 0.845 | Val Loss: 0.2077 Acc: 0.924
Epoca [25/25] Train Loss: 0.3538 Acc: 0.858 | Val Loss: 0.2003 Acc: 0.926
Mejor Val Acc: 0.931

```

VolleyballCNN -- Entrenamiento



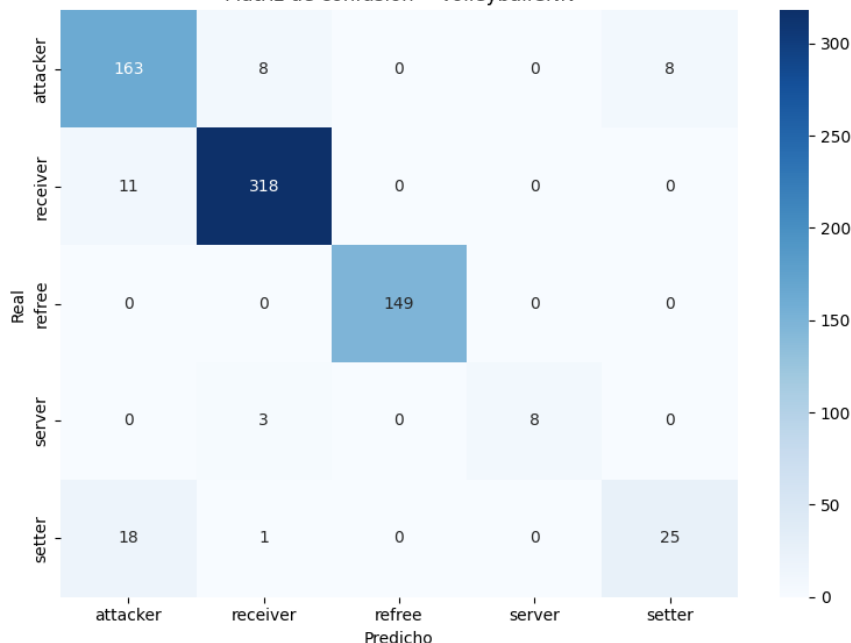
```

ActionDataset "valid": 712 crops
precision recall f1-score support
attacker      0.85      0.91      0.88      179
receiver       0.96      0.97      0.97      329
refree         1.00      1.00      1.00      149
server         1.00      0.73      0.84       11
setter         0.76      0.57      0.65       44

accuracy      0.93      0.93      0.93      712
macro avg     0.91      0.83      0.87      712
weighted avg  0.93      0.93      0.93      712

```

Matriz de confusion -- VolleyballCNN



3.4 Trajectory Prediction (LSTM)

Player movement is modelled by a two-layer bidirectional LSTM (hidden size 128, about 0.55 M parameters). It reads a window of 20 past frames, each described by five values: normalised position x and y, velocity vx and vy, and detection confidence. It predicts the next 10 positions.

Two design decisions are worth explaining. First, the network predicts displacement relative to the last observed position rather than absolute coordinates. With a displacement target the naive "stay put" baseline is simply zero, so the model only looks good if it actually learns real motion. Second, we split trajectories temporally (first 80% trains, last 20% validates) rather than splitting windows at random, which would leak almost-identical overlapping windows into both sets.

We extracted trajectories from the match video using the player detector and ByteTrack, keeping only tracks of at least 30 frames and sub-sampling one frame in four. This gave 468 training windows and 56 validation windows. Training used MSE loss, Adam (lr=1e-3) and a step schedule over 50 epochs. The best validation MSE was 0.00105, against a naive baseline of 0.00116, so the LSTM beats "stay put" by about 10%.

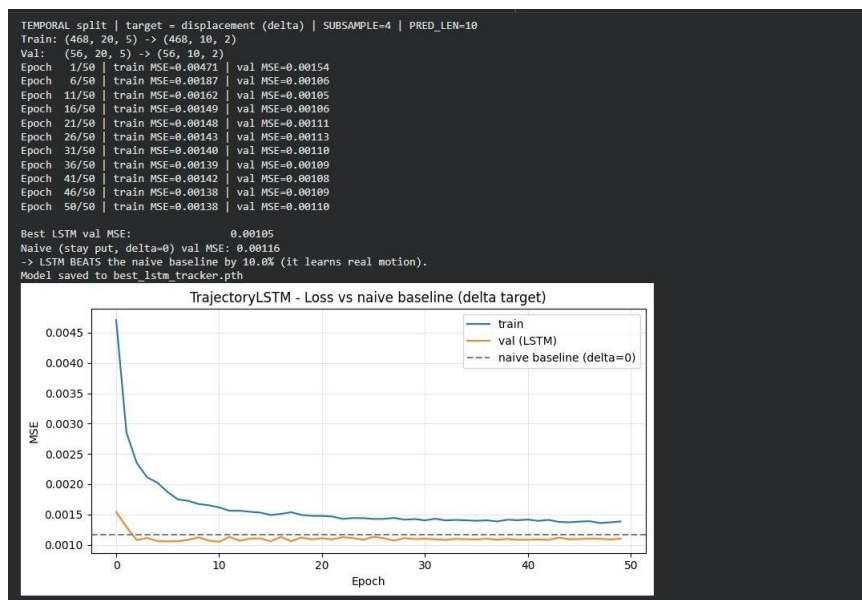


Figure 4. LSTM training: loss vs. naive baseline over 50 epochs. Best val MSE = 0.00105.

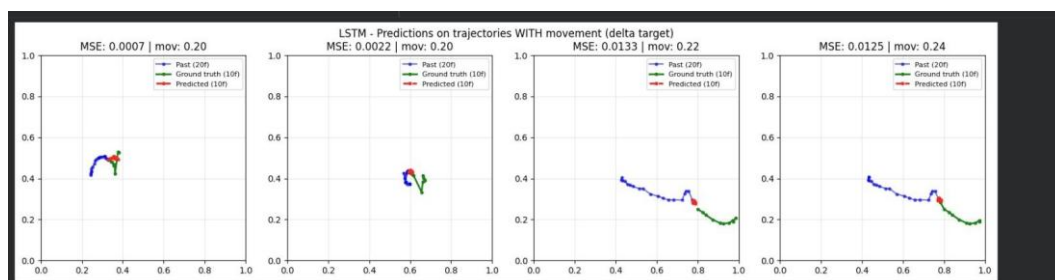


Figure 5. LSTM trajectory predictions on validation sequences. Blue: past 20 frames. Green: ground truth next 10. Red: model prediction.

3.5 Team Classifier (MLP)

Assigning a player to team 1, team 2 or referee uses TeamMLP, a small fully-connected network (16 -> 32 -> 16 -> 3, only 1,123 parameters). The input is a 16-bin HSV colour histogram of the player's torso region (8 bins for the H channel and 8 for S).

Rather than label the data by hand, we ran K-means with $k=2$ over the torso histograms of all detected players across the video to discover the two team colours automatically. Players inside predefined referee zones are labelled as the third class. This produced 764 samples (210 team 1, 493 team 2, 61 referee), so we trained with class weights [1.21, 0.52, 4.17] to handle the imbalance. Training used AdamW and cross-entropy loss over 40 epochs. Best validation accuracy was 90.8%. In the pipeline the homography side of the net takes precedence, and the MLP overrides that decision only when its confidence is above 65%.

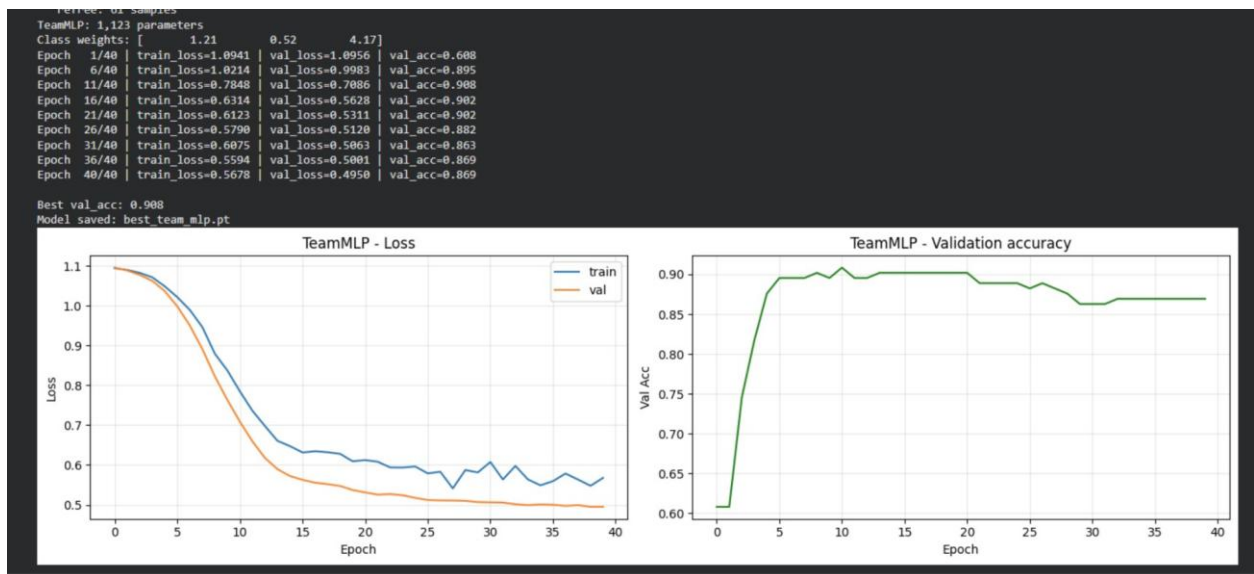


Figure 6. TeamMLP training curves over 40 epochs. Best val accuracy = 90.8%.

Figure 7. TeamMLP

3.6 Vision-Language Assistant (Gemma 4)

The language layer uses Gemma 4 (unsloth/gemma-4-E2B-it) loaded through Unsloth, which keeps memory use low enough to run alongside the rest of the pipeline on a single T4 GPU. We did not fine-tune it. Instead, the pipeline first analyses the frame and builds a structured summary: player count per team, ball position, per-player role and court side. That summary goes to Gemma together with the annotated image, so the model answers questions using both the picture and the system's own detections rather than guessing from the image alone.

3.7 Integration and Interface

All components are wired together in the VolleyballPipeline class and exposed through a Gradio interface with four tabs: image processing, video processing, the Gemma assistant, and CNN retraining. The homography is corrected per frame using ORB + RANSAC on the stands region so the minimap projection stays aligned when the camera pans. Player positions are projected through their foot point rather than the bounding box centre, which gives a more accurate ground-plane position on the minimap.

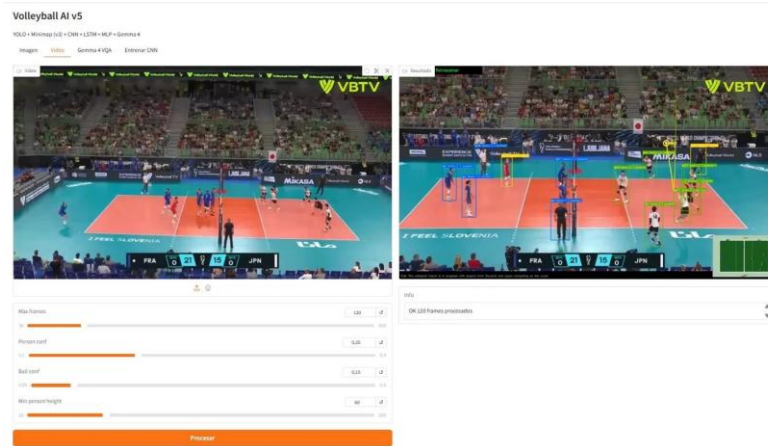


Figure 8. Gradio video tab: input clip (left) and annotated result with minimap (right). 120 frames processed, VLM description at the bottom.



Figure 9. Gemma 4 VQA tab: user asks "what sport is being played?" and the model responds using both the annotated frame and the pipeline detection context.



Figure 10. Train CNN tab: configurable epochs, learning rate and batch size. The CNN can be retrained from the interface without touching the code.

4. Datasets and Training Notebooks

Each model was trained in its own Colab notebook. The main application (v12_final.ipynb) loads the weights and trains the team MLP and optionally the role CNN from inside the interface.

Dataset	Used for	Source
player.yolov8	Player detector	Public, Roboflow
volleyball-ball-a-jpn.yolov8	Ball detector	Built by us from match video
volley_detection v5	Role CNN (player crops)	Public, Roboflow (hyunwoon am)
Match video (FRA vs JPN)	LSTM trajectories + MLP team labels	Our own recording

Table 2. Datasets used per module.

Notebook	Produces
TrainPersonVF.ipynb	volleyball_persons_best.pt
BallTrain_VF.ipynb	volleyball_ball_best.pt
TrainLSTM_temporal_Final.ipynb	best_lstm_tracker.pth
train_cnn_classifier.ipynb	best_cnn_classifier.pth
v12_final.ipynb	Main application + TeamMLP training + GUI

Table 3. Training notebooks and their outputs.

5. How to Run

Open v12_final.ipynb in Google Colab with a GPU runtime (T4 or better). Run the sections in order: Section 0 installs dependencies (Gemma 4 / Unsloth takes two to three minutes). Sections 1 to 8 define all constants, architectures and helper functions. Section 9 loads the models: upload best_cnn_classifier.pth, best_lstm_tracker.pth, volleyball_persons_best.pt and volleyball_ball_best.pt before running it. Section 10 applies patches for referee detection and minimap rendering. Sections 11 and 12 train and integrate the team MLP from the video. Section 13 runs a single-frame test. Section 14 launches the Gradio interface.

For each model you can either upload the weight file or train it. The player and ball detectors are trained only in their own notebooks and just loaded in the main app. The role CNN can be retrained from the "Train CNN" tab inside the interface (the volley_detection-5 dataset needs to be in /content/). The team MLP is always retrained from the video during startup.

6. Design Decisions

The most important decision was to build our own ball dataset. The standard COCO sports-ball class missed the ball on almost every frame of our footage. Extracting and annotating frames from the match video was the only way to get a detector that works on our specific recording.

For the players we reused an existing Roboflow dataset because labelled volleyball players were already available. Training the LSTM and the team MLP on our own match video was deliberate: the camera angle, court and lighting at inference time are exactly the ones the models saw during training.

The displacement target and temporal split for the LSTM were chosen so the reported numbers are honest. An absolute-position target would produce a much lower MSE that mostly reflects repeating the last position rather than real prediction. The temporal split prevents overlapping windows from leaking between sets.

For team assignment we combine geometry and colour: the homography places each player on one side of the net, and the MLP overrides when confident. This is more robust than either signal alone, especially when the camera pans and the homography drifts slightly.

We used Claude as a coding assistant throughout the project, mainly to translate notebook comments and interface to English and to help structure this report. Every change was checked by running the affected part of the pipeline.

7. Conclusions and Lessons Learned

The system works end to end. From a raw clip it produces tracked players with team and role labels, a ball track, trajectory arrows, a minimap, and a language assistant, all from one interface. The part we are most satisfied with is the ball detector, because solving it required recognising that the public model was not good enough and building our own data.

The clearest limitation is the size of the validation sets, especially for the ball, where the small number of annotated frames makes the strict mAP@50-95 metric unreliable. The LSTM gain over the naive baseline is real but modest, which partly reflects the nature of short-window volleyball motion and partly the fact that we only had one match for sequence data. The referee class in the MLP confusion matrix shows zero recall in the colour-only evaluation, which is expected because referees are identified by court zone rather than shirt colour.

If we did this again we would annotate more frames for both detectors, gather trajectory data from several matches, and consider fine-tuning Gemma on volleyball captions instead of relying on prompting alone.

Individual Contributions

Angel worked on the vision and detection side: the player and ball YOLO detectors, dataset preparation and visual metrics, the role-classification CNN, and the colour-based team MLP with its K-means labelling.

Carlos worked on the sequence and tracking side: the ByteTrack integration, trajectory extraction, and the design, training and evaluation of the LSTM, including the displacement target and the temporal-split methodology.

Javier and Jorge worked on integration, interface and language: the Gradio GUI, the Gemma 4 assistant and its context prompting, the homography, minimap and camera-motion correction, and

the full pipeline assembly together with the repository and documentation.

Architecture decisions, sport and dataset selection, and final system cross-checking were done jointly. All members contributed to all parts of the project.